ATmega32A Module
Interrupts

Ali Sahafi

1 Theory

Polling or interrupt? With *polling* the program asks again and again: "Is the ADC finished? Has the timer overflowed? Has a byte arrived?" The CPU spends its time waiting. With an *interrupt* the peripheral tells the CPU itself when something happens. Until then the CPU is free to do other work.

What happens at an interrupt.

1. An event happens (button edge, timer compare match, ADC finished, byte received...) and the peripheral sets its *flag*.
2. If this interrupt is enabled *and* the global interrupt switch is on (`sei()`), the CPU finishes the current instruction.
3. The CPU saves where it was and jumps to the *Interrupt Service Routine* (ISR) of this event. While the ISR runs, other interrupts wait.
4. At the end of the ISR the CPU goes back to the main program, exactly where it stopped.

Vector table and priority. The ATmega32A has 21 interrupt sources. Each one has a fixed address in the *interrupt vector table*. If two interrupts come at the same time, the one with the lower address goes first (`INT0` has the highest priority).

Rules for interrupt functions.

- Keep them **short**: no `_delay_ms()`, no long loops. Other interrupts wait while one runs.
- A variable that the interrupt function changes and `main()` reads must be **`volatile`**, for example `volatile uint8_t count = 0;` Without `volatile` the compiler may never read the new value in `main()`.

2 How to use the drivers

In the drivers, your interrupt function is a *normal function*. You give it to the driver, and the driver writes the ISR for you, enables the interrupt and calls `sei()`. All interrupt functions of the course drivers:

Driver function	ISR (vector)	Your function is called ...
`ExternalInterrupt.enable(INT0, when, f)`	`INT0_vect`	on an edge of `PD2` (button S11). Also `INT1`, `INT2`
`Timer0.onOverflow(f)`	`TIMER0_OVF_vect`	when Timer0 overflows
`Timer0.onCompareMatch(f)`	`TIMER0_COMP_vect`	on a compare match (Timer1/Timer2: the same)
`ADC.onComplete(f)`	`ADC_vect`	when a conversion is finished: `f(value)`, value 0–1023. Start a conversion with `ADC.start(channel)`
`UART.onReceive(f)`	`USART_RXC_vect`	when a byte has arrived: `f(byte)`
`sei()` / `cli()`		switch *all* interrupts on / off (global switch)

Pass `nullptr` instead of a function to switch an interrupt off again (`ExternalInterrupt.disable(INT0)` for the external interrupts).

External interrupts

```
ExternalInterrupt.enable(pin, when, function);        // no debounce
ExternalInterrupt.enable(pin, when, function, debounceMs); // for a push button
ExternalInterrupt.disable(pin);
```

Argument	Value	Meaning
`pin`	`INT0`	pin `PD2`: push button **S11** on the board
	`INT1`	pin `PD3` (free pin on the header)
	`INT2`	pin `PB2`: this is LED D2 on the board, so you can not use both
`when`	`INT_FALLING`	HIGH → LOW: the button is *pressed*
	`INT_RISING`	LOW → HIGH: the button is *released*
	`INT_ANY_EDGE`	both edges (`INT0`, `INT1` only)
	`INT_LOW_LEVEL`	again and again, as long as the pin is LOW (`INT0`, `INT1` only)
`debounceMs`	`0` (default)	no debounce: every edge calls your function
	`1`–`255`	debounce time in ms, for example `20` for a push button

`enable()` makes the pin an input with pull-up. A wrong pin or edge is a **compile error**, for example `ExternalInterrupt.enable(PD2, ...)` (you must write `INT0`, not `PD2`).

Button bounce. The push buttons on the board have no hardware debouncer. When the metal contacts close, they bounce a few times, so one press can make *several* edges. We tested it on the board: 5 presses of S11 gave the counter 7 without debounce. With `debounceMs = 20`, 10 presses gave exactly 10. With a debounce time the driver waits after the first edge, forgets the extra edges, and calls your function only if the pin is still at the new level (for `INT_FALLING`: still pressed). During this wait all other interrupts must wait too, so use it only for buttons, not for fast signals.

Driver functions and registers

Driver function	Registers

`ExternalInterrupt.enable(INT0, INT_RISING, f)`	`DDRD &= ~(1<<PD2); PORTD |= (1<<PD2);`
	`MCUCR |= (1<<ISC01) | (1<<ISC00);` (rising edge)
	`GIFR = (1<<INTF0); GICR |= (1<<INT0); sei();`

`ExternalInterrupt.disable(INT0)`	`GICR &= ~(1<<INT0);`
`Timer0.onCompareMatch(f)`	`TIMSK |= (1<<OCIE0); sei();`
`Timer0.onOverflow(f)`	`TIMSK |= (1<<TOIE0); sei();`
`ADC.onComplete(f)`	`ADCSRA |= (1<<ADIE); sei();`
`ADC.start(0)`	`ADMUX = (ADMUX & 0xE0) | 0; ADCSRA |= (1<<ADSC);`
`UART.onReceive(f)`	`UCSRB |= (1<<RXCIE); sei();`

3 Example: up counter with button S11

Every time button S11 is released, the interrupt adds 1 to the counter on the LEDs. The main loop is empty. Try it and watch the bounce: sometimes the counter jumps by 2 or 3. Then add a debounce time of 20 ms as the fourth argument of `enable()` and try again.

Build and upload. `make interrupt`

```
/*
 * External Interrupt Example -- an up counter on the LEDs, button S11
 * Build & flash: make interrupt
 *
 * Every time the push button S11 (PD2 = INT0) is released, the interrupt
 * adds 1 to the counter. The main loop does nothing: the counting happens
 * only in the interrupt function.
 *
 * Watch out: a mechanical button "bounces". One press can make more than
 * one edge, so the counter sometimes jumps by 2 or 3.
 */

#define F_CPU 8000000UL
#include "../gpio/gpio.hpp"
#include "interrupt.hpp"

volatile uint8_t count = 0; // volatile: changed in the interrupt function

void onRelease() {       // runs automatically on every rising edge of PD2
  count++;
  GPIO.write(PORTB, ~count); // LEDs are active-low
}

int main() {
  GPIO.setDirection(PORTB, ALL, OUTPUT);
  GPIO.write(PORTB, 0xFF); // all LEDs off

  ExternalInterrupt.enable(INT0, INT_RISING, onRelease); // also calls sei()

  while (true) {
    // nothing to do here -- the interrupt does the work
  }
}
```

4 Laboratory Tasks

For each task, make a new `.cpp` file in the main project folder (for example `Task1.cpp`). Start the file with `#define F_CPU 8000000UL` and include the drivers you need. Upload it with `make SRC=Task1.cpp`. In all tasks the main loop `while (true) { }` stays **empty**: all the work is done in interrupt functions.

Task 1: Real-Time Clock (Timer0 Interrupt)

Count 0 to 9 on the seven-segment display, one step every *exactly* 1 second (external 8 MHz crystal). Use `Timer0` in `TIMER_CTC` mode with prescaler 8 and a compare match interrupt:

1. With prescaler 8, one count takes 1 μs. Choose the compare value so that there is one compare match every 200 μs: `Timer0.setCompare(...)`.
2. Give your tick function to `Timer0.onCompareMatch()`.
3. In the tick function, count the ticks. How many ticks are 1 second? Then show the next digit with `SevenSeg(digit)`.

Use `static` or `volatile` variables for the counters.

Task 2: ADC Interrupt

Show the potentiometer on the LEDs, using the ADC interrupt instead of waiting:

1. `ADC.enable()`, give your function to `ADC.onComplete()`, then start the first conversion with `ADC.start(0)`.
2. Your function gets the value 0–1023. Show the upper 8 bits on the LEDs (`value >> 2`, active-low), then start the next conversion with `ADC.start(0)`.

Hint: do not use `ADC.read()`, `ADC.read8()` or `ADC.readMillivolts()`: they are *polling* (they start a conversion and wait until it is finished).

Task 3: UART Receive Interrupt

Show every byte that the PC sends on the LEDs on `PORTB`. Give your function to `UART.onReceive()`: it gets the received byte. Compare with Lesson 6, Task 1.2: where does the program wait now?

Task 4: Potentiometer Logger (Timer + ADC Interrupts)

Measure the potentiometer (`PA0`) *exactly* one time per second and send the voltage in millivolts to the PC with the UART (9600 baud), for example `U = 2500 mV`.

- A **timer interrupt** decides *when* to measure. Which timer, mode, prescaler and compare value give you 1 second?
- The **ADC interrupt** gives you the value 0–1023. **Hint:** do not use `ADC.read()`, `ADC.read8()` or `ADC.readMillivolts()`: they are *polling* (they wait for the conversion). Calculate the millivolts yourself, in the same way as `readMillivolts()` does (see the ADC handout).
- Sending text takes milliseconds: do not send from an interrupt function. Here the main loop is not empty: it only sends.

Assignment 1 (the ultrasound sensor) is described on the lecture slides.

Assignment 2: LED Brightness from the PC

Receive bytes from the PC with `UART.onReceive()` and use each byte as the brightness of LED D3: `Timer0` in `TIMER_FAST_PWM` mode with `PWM_INVERTING`, and `Timer0.setCompare(byte)` in your receive function.

Assignment 3: Measurements on Command

The PC sends a command of **two bytes**: the first byte is the *number* of measurements, the second byte is the *time* between them in milliseconds. The board then measures the potentiometer this number of times and sends every value back with `UART.println()`.

- Receive the two bytes with `UART.onReceive()`. Store them in `volatile` variables and set a `volatile` flag when both bytes have arrived.
- In `main()`, when the flag is set, do the measurements. (Here the main loop is not empty: long work does not belong in an interrupt function.)